

Methods and Statistics: Presentation Companion

Seman Libbiss and Paz Flashner
Questions, pipeline details and speaking explanations

This companion contains the detailed material removed from the presentation. The presentation itself contains only the ten main slides. Use these sections to prepare answers; the main report and its appendix provide the formal account and numerical tables.

1. Resampling and the two time windows

Model input: target motion to final tracker sample.

Feature input: detected finger onset to end.

A phase of 0.5 means half the selected time interval, not half the path length.

$$\mathbf{P}_j = \mathbf{S}(j/99) - \mathbf{S}(0), \quad j = 0, \dots, 99.$$

100 samples are a common representation, not 100 independent observations.

How to explain it

We start with a regularized 240 Hz grid, smooth using a fourth-order 10 Hz forward-backward Butterworth filter and select the modeled window. We fit a cubic interpolant through the existing filtered points and evaluate it at $j/99$. This interpolation is different from fitting the later five-knot compression model. Subtracting the first position aligns starts, but does not scale amplitude or rotate paths. We retain initiation and movement seconds as separate targets. Early movement after target motion remains; the earlier appearance interval is outside the modeled window. Source: `src/preprocessing.py`, `src/trajectory_view.py`; appendix A1.

2. What becomes the PCA matrix?

Five interior knots + degree 3 + 1 = nine coefficients per axis.

Object	Dimensions	One row / one column means
Sampled trial	100 x 2	Time point / x or y
Spline basis B	100 x 9	Time point / basis function
Trial coefficient vector	18	Nine x coefficients followed by nine y
Training coefficient matrix	N training x 18	Trial / coefficient
PCA directions V	18 x n	Coefficient / retained component
Training latent codes	N training x n	Trial / latent coordinate

How to explain it

Moni asked what we feed into PCA. It is a matrix where each row is one trial's eighteen spline coefficients. We first fit a spline separately to x and y of each trial. Nine x and nine y coefficients give eighteen numbers. We stack training trials and PCA learns n directions in that eighteen-dimensional space. Neither test trials nor validation trials train those PCA directions. The number of knots and the number of latent coordinates are different controls. Source: `src/baseline_spline.py`, `CompactSplineRepresentation`; appendix A2.

3. Fitting and decoding spline + PCA

Fit the two coordinate coefficient vectors by least squares.

Validation selects raw or standardized coefficients.

Inverse PCA gives coefficients; multiplying by B restores the path.

$$\mathbf{z} = V^T[(\mathbf{c} - \mathbf{m}) \oslash \mathbf{d}], \quad \hat{\mathbf{c}} = \mathbf{m} + \mathbf{d} \odot (V\mathbf{z}).$$

m and d combine centering/scaling; V contains retained PCA directions. Raw PCA uses d = 1.

How to explain it

There are two compression steps: sampled path into smooth coefficients, then coefficients into a smaller PCA code. Fit B c_x to x and B c_y to y. The code z is obtained by centering/scaling the coefficient vector and projecting onto retained axes. To decode, apply inverse PCA and inverse scaling, split the eighteen coefficients into two blocks and multiply both by the basis B. Raw PCA still centers columns. PCA optimizes variance in coefficient space and does not guarantee the best trajectory MSE against all methods. Sources: src/baseline_spline.py; appendix A2. The equation above matches the effective forward and inverse transformations.

4. Predicting physical timing

Observed timing never enters the trajectory encoder. Reconstruction-time timing prediction sees the observed shape.

Stage	Spline + PCA	Neural models
Predictors	n PCA scores + 5 conditions	Latent code + conditions where enabled
Targets	Two standardized $\log(t + .001)$ values	Same two standardized targets
Predictor	Ridge, alpha selected on validation	Separate decoder output head
Inverse	Undo scaling, exponentiate, subtract .001	Same inverse transform

How to explain it

The coordinate path is on normalized phase, so the model must predict seconds separately. Initiation is time from target motion to finger onset, movement time is onset to end. We take $\log(t+.001)$ to accommodate zeros, standardize training targets, and reverse these transformations after prediction. Spline Ridge receives the code and five conditions, with training input standardization and alpha from .01, .1, 1, 10, 100 selected using validation error on standardized log targets. Neural models have a two-value timing head. Huge positive timing errors were not upper-clipped out of the main scores. Source: `src/confirmatory_spline.py`, `TimingRidge`; `src/vae_model.py`; appendix A3.

5. The neural training objective

Coordinate error sums 200 squared differences.

Timing error sums two squared differences and receives weight 20.

KL sums across latent dimensions; the batch averages the total.

$$\mathcal{L} = \|\mathbf{v} - \hat{\mathbf{v}}\|_2^2 + 20\|\mathbf{y} - \hat{\mathbf{y}}\|_2^2 + \beta D_{\text{KL}}[q(\mathbf{z} | \mathbf{v}, \mathbf{c}) \| \mathcal{N}(\mathbf{0}, I)].$$

Replacing both summed terms by separate MSE means would change the effective timing weight.

How to explain it

The actual code uses `reduction=None`, then sum across output coordinates and mean across trials. We must not simply write MSE plus 20 MSE because one output has two hundred coordinates and the other has two. KL measures how far the encoder's Gaussian posterior is from the standard normal. Beta warms from zero to one over fifty epochs, while checkpoint validation uses the final beta-one objective. Adam learning rate .001, batch size 64, gradient norm clipping at five, patience 25, maximum 150 epochs, three seeds. Source: `src/vae_model.py` `vae_loss`; `src/train.py`; appendix A3.

6. VAE, CVAE and CAE

A nonsignificant conditional-model advantage does not mean the experimental conditions are irrelevant.

Property	VAE	CVAE	CAE
Trajectory encoder	Two hidden layers	Two hidden layers	Two hidden layers
Task inputs	Masked to zero	Encoder and decoder	Encoder and decoder
Latent training code	Gaussian sample	Gaussian sample	Deterministic
KL penalty	Yes	Yes	No
Reconstruction code	Posterior mean	Posterior mean	Deterministic code

How to explain it

Our VAE is an unconditional ablation in the same code framework. It does not receive meaningful conditions because we zero them at both ends. CVAE uses actual conditions. CAE uses actual conditions but suppresses probabilistic sampling and KL. The condition vector is three one-hot category entries, a side value (left zero, right one), and normalized executed speed. We checked input routes and shuffled conditions. The routes are active, but these results cannot settle whether another architecture or training regime would use conditions better. Source: `src/vae_model.py` `encode_condition/encode_trial_condition`; `src/train.py`; audit records.

7. Where the generated variation comes from

The personal center averages context codes.

Training residuals estimate within-person latent variation.

Variational models also add average posterior variance.

$$\Sigma_{\text{train}} = \text{Cov}_{\text{train}}(\mu_i - \bar{\mu}_{s(i)}) + \text{diag}(\mathbb{E}_{\text{train}}[\sigma_i^2]) + 10^{-6}I.$$

One shared covariance for every participant; 120 draws around each personal center.

How to explain it

For each training person, subtract that person's mean code from each trial code. Pool these residuals and estimate their covariance. Add the mean encoder posterior variance on the diagonal for VAE/CVAE and add a tiny identity regularizer for numerical stability. CAE and spline have no posterior variance term. We do not fit a separate covariance to a new participant's context trials. Generation draws a multivariate normal around their context center and uses the shared decoder. A nonlinear decoder can still produce different visible spread at different centers. Source: `src/context_query.py` and `src/confirmatory_spline.py`; appendix A4.

8. Leakage and held-out evaluation

Population model fitting and participant enrollment are separate stages.

Information	Training	Validation	Test context	Test query
Fit network/PCA/scalers	Yes	No	No	No
Select checkpoint/settings	No	Yes	No	No
Personal center	No	No	Yes	No
Reconstruction input	No	No	No	Observed path
Generation task mixture	No	No	No	Metadata only
Generation scoring	No	No	No	Paths and timings

How to explain it

A test person is new to model fitting, but provides context recordings. Query recordings only provide the targets for generation scores, with task metadata allowed for a matched-condition mixture. Reconstruction deliberately reads the query path because that task is compression. None of the test data changes fitted PCA axes, learned neural parameters, training covariance or normalization. This prevents a common misunderstanding: test context is not retraining. Model selection for the final dashboard still used the evaluated cohort, so independent post-selection validation remains future work. Source: participant_folds.json, context_query.py; appendices A1/A4.

9. What the eleven features measure

Generated feature computation uses predicted timing and an onset-based movement crop.

Group	Features
Timing	Initiation time; movement time; relative time to peak speed
Speed	Peak speed; heuristic speed-peak count
Geometry	Path length; straight-line distance; curvature ratio; maximum lateral deviation
Endpoint	Final x; final y

How to explain it

For observed data we use the onset-to-end movement segment. For generated data we use the predicted initiation fraction to crop the go-to-end path, capped at .98, and floor movement duration at one millisecond. Both movement segments become one hundred phase points and are filtered with cutoff $\min(10 \text{ Hz}, .45 \text{ times sample rate})$. Sample rate is 99 over movement seconds. Gradients give speed. Peak count uses prominence ten percent of maximum and roughly fifty milliseconds separation, with at least one reported peak. This is distinct from minimum-jerk component fitting. Source: `src/features.py`; appendix A5.

10. KS and joint distribution distances

KS: largest gap between two empirical cumulative curves.

Mean KS: average of eleven feature statistics.

Energy and MMD: compare whole vectors and account for joint structure.

$$D_{sf} = \sup_x |F_{sf}(x) - G_{sf}(x)|, \quad \overline{D}_s = \frac{1}{11} \sum_{f=1}^{11} D_{sf}.$$

KS statistics are not p-values. Correlated features can affect joint geometry.

How to explain it

If a recorded feature has seventy percent of trials below a value and the generated feature has forty percent, the empirical CDF gap there is .3. KS takes the largest gap over all thresholds. Its output is a discrepancy. We average discrepancies across features, not significance tests. Energy and MMD use complete feature vectors and can notice dependence between features that separate marginal KS statistics may miss. Standardization comes from training. Correlated features do not mean we should remove whichever feature makes a p-value improve; removal defines a new endpoint. Sources: `src/context_query.py`; appendix A5/A6; Gretton et al. JMLR 2012.

11. Energy discrepancy and mean collapse

Energy compares cross-set distances with within-set distances.
A generator needs plausible variation as well as a good center.

$$E = 2 \overline{\|X - Y\|} - \overline{\|X - X'\|} - \overline{\|Y - Y'\|}.$$

Lower is better. The mean-only output has a closer mean but a much worse distribution.

Example reference	Mean-path RMSE	Path energy
VAE n=8	0.1604	0.0548
Repeated context mean	0.1146	0.4621

How to explain it

Average error against all recorded paths can favor a collapsed output near the center. Energy subtracts within-generated and within-recorded distances from twice the cross average, addressing the distributions rather than only their centers. Our energy estimator includes within-sample diagonal zeros and is not square-rooted afterward. The mean-only reference is actual analysis output, not a hypothetical numerical illustration. Source: `src/trajectory_distribution.py`; `trajectory_distribution` summary CSV; Szekely and Rizzo 2013; appendix A6/A8.

12. Gaussian-kernel MMD

Kernel: $\exp(-\gamma \times \text{squared Euclidean distance})$.

Gamma comes from the median positive training pair distance.

Within-set diagonals are excluded in the unbiased estimator.

$$\widehat{\text{MMD}}_u^2 = \frac{\sum_{i \neq j} k(X_i, X_j)}{m(m-1)} + \frac{\sum_{i \neq j} k(Y_i, Y_j)}{r(r-1)} - \frac{2 \sum_{i,j} k(X_i, Y_j)}{mr}.$$

Finite-sample unbiased MMD squared may be slightly negative. It is not an accuracy percentage.

How to explain it

The kernel gives high similarity to close vectors and low similarity to distant vectors. MMD compares average within-recorded and within-generated similarity with cross similarity. The within terms omit identical-element pairs and divide by $m(m-1)$ or $r(r-1)$; cross divides by mr . We estimate gamma once from at most 512 deterministic training trials in each representation and use it for all models in the rotation. Whole paths and feature vectors have different units and scales, so their absolute MMD values are not interchangeable. Source: `src/trajectory_distribution.py`; `src/context_query.py`; Gretton et al., A Kernel Two-Sample Test, JMLR 13 (2012).

13. 28 paired scores, not thousands of independent trials

First average neural seeds within each participant.

Then compare paired participant scores using two-sided Pratt Wilcoxon.

Round differences to 12 decimals before ranking.

BH controls false discoveries under its assumptions. Holm is a familywise-error sensitivity. Nonsignificance is not equivalence.

Correction family	Tests
Original seven endpoints $\times$ ten pairs $\times$ two dimensions	140
New two path endpoints $\times$ ten pairs $\times$ two dimensions $\times$ two geometries	80
VAE replacement controls	6
CVAE replacement controls	12

How to explain it

Each participant supplies one score per model for the paired comparison. Three seeds are repeated model fits, not three independent people. Wilcoxon ranks the absolute paired differences and uses their signs. Pratt keeps zeros in ranking but omits their signed contribution. The usual location interpretation assumes a symmetric difference distribution. BH adjusts the complete declared family, not only the comparisons we liked. We additionally show Holm. Training folds overlap, so these remain exploratory comparisons, and a fresh cohort is still needed for replication. No formal three-versus-eight or resampling-reference test was added. Source: `src/statistical_tests.py`, `all_pairs_140.csv`, `paired_comparisons.csv`; appendix A6.

14. The boundaries of the conclusion

Useful same-session personal centers do not establish stable traits.

Direct context summaries beat latent linear probes on all 14 tested targets.

Condition routes work, but their value depends on the task and model.

Minimum-jerk decomposition remains a separate investigation.

The 31-page laboratory report retains the extended results and Jason's open questions.

How to explain it

We should volunteer the relevant limitation instead of defending every result as a success. Direct context means and standard deviations were more accurate than our latent linear probes, and many probe R-squared values were negative. Matched minimum-jerk component comparisons did not establish corrected model differences. Timing and event assumptions remain important, including the tracker endpoint arriving on average 26 ms before the MATLAB arrival marker. More data might help, but we did not run a learning curve that proves sample count caused the failures. Source: frozen report pp.16–28, production/all_models_direct_context_summary.csv; course Discussion and appendix A10.

15. Launch and source navigation

Repository terminal:

```
python -m streamlit run src/confirmatory_dashboard.py --server.port 8510
```

Browser: <http://127.0.0.1:8510>

Stop: Ctrl+C in that terminal

The standalone dashboard archive contains launch instructions for use without the full research checkout.

How to explain it

Full PowerShell path: `cd "D:\oneDrive_Main\OneDrive\Desktop_onedrive\uni2024\Third Year\semester B\Workshop in AI\interception-movements-dl"`. Then use the command shown, adding `--server.address 127.0.0.1 --browser.gatherUsageStats false` if desired. On the authoring computer Python is `C:\Python313\python.exe`. Source navigation: `src/vae_model.py` and `src/train.py` for neural models; `src/baseline_spline.py` and `src/confirmatory_spline.py` for spline/PCA/Ridge; `src/context_query.py` and `src/features.py` for generation/metrics; `src/trajectory_distribution.py` for full paths; `production/audit_2026_09_08` for original result CSVs; `production/trajectory_distribution_2026_09_13` for new results. Keep the terminal running during the demo.

16. What changed after checking feature correlations?

Training-only feature selection; complete 72-test sensitivity family.

How to explain it

We recorded a priority and a 0.90 correlation threshold before computing the results. A feature needed strong pooled and median within-participant Spearman correlation with a retained feature. All four training rotations dropped straight-line distance and endpoint y in favor of path length. With nine features, both VAE and CVAE still beat spline on all three metrics at both capacities under BH and Holm. VAE versus CVAE at eight dimensions no longer passed BH on any reduced metric: $q=.149$ for KS, $.289$ for energy and $.428$ for MMD. We therefore do not call VAE uniquely best. Removing features changes what we measure; it does not automatically reduce our correction family, because mean KS is one score, not eleven feature p-values. Original and reduced views were corrected together across six pairs, two capacities and three metrics: 72 tests. See report Appendix A11 and [production/feature_redundancy_2026_09_14/](#).